

并行程序的执行速度可能会因程序的资源需求与硬件资源约束之间的交互而大大不同。管理并行代码与硬件资源约束之间的交互对于在几乎所有并行编程模型中实现高性能都至关重要。这是一种实用技能，需要对硬件架构有深入的理解，并最好通过针对高性能设计的并行编程模型进行实践练习来学习。

到目前为止，我们已经了解了 GPU 架构的各个方面以及它们对性能的影响。在第四章《计算架构和调度》中，我们学习了 GPU 的计算架构及相关性能考虑因素，如控制分歧和占用率。在第五章《内存架构和数据局部性》中，我们了解了 GPU 的片上内存架构以及使用共享内存瓦片化来实现更多的数据重用。在本章中，我们将简要介绍片外内存 (DRAM) 架构，并讨论相关的性能考虑因素，如内存合并和内存延迟隐藏。然后，我们讨论了一种重要的优化类型——线程粒度粗化(thread granularity coarsening)——它可以针对架构的不同方面进行优化，具体取决于应用程序。最后，我们将本书的这一部分总结为一个常见性能优化的清单，作为优化将在本书第二和第三部分中讨论的并行模式性能的指南。

在不同的应用程序中，不同的架构约束可能会主导并成为性能的限制因素，通常被称为瓶颈。通过在某个 CUDA 设备上交换一个资源的使用来显著提高应用程序的性能是可行的。如果这种策略减轻的资源约束在应用策略之前是主导约束，并且加重的约束不会对并行执行产生负面影响，则此策略效果良好。如果没有这样的理解，性能调优将是一种猜测；合理的策略可能会或可能不会导致性能提升。

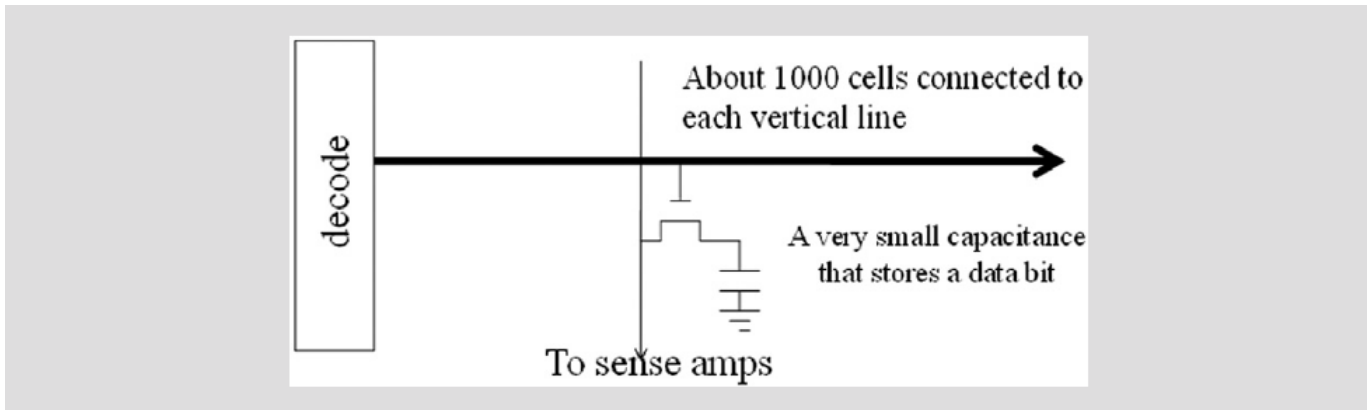
6.1 内存合并(Memory coalescing)

CUDA 内核性能的最重要因素之一是访问全局内存，其带宽有限可能成为瓶颈。CUDA 应用程序广泛利用数据并行性。自然地，CUDA 应用程序倾向于在短时间内处理大量来自全局内存的数据。在第五章《内存架构和数据局部性》中，我们学习了利用共享内存的瓦片化技术，通过每个线程块中的一组线程减少必须从全局内存访问的总数据量。在本章中，我们将进一步讨论内存合并技术，以有效地在全局内存和共享内存或寄存器之间移动数据。内存合并技术通常与瓦片化技术结合使用，以允许 CUDA 设备通过有效利用全局内存带宽达到其性能潜力。(最近的 CUDA 设备使用芯片内缓存来存储全局内存数据。这种缓存会自动合并更多的内核访问模式，从而在一定程度上减少了程序员手动重新排列其访问模式的需求。然而，即使有了缓存，在可预见的未来，合并技术仍将对内核执行性能产生显著影响。)

CUDA 设备的全局内存是使用 DRAM 实现的。数据位存储在小电容器的 DRAM 单元中，其中微小电荷的存在或缺失区分了 1 和 0 的值。从 DRAM 单元读取数据需要小电容器利用其微小电荷驱动高电容线到达传感器，并触发传感器的检测机制，以确定电容器中是否存在足够的电荷来合格为“1”。这个过程在现代 DRAM 芯片中需要数十纳秒（见“为什么 DRAM 如此慢？”侧栏）。这与现代计算设备的亚纳秒时钟周期时间形成鲜明对比。由于这个过程相对于所需的数据访问速度非常慢（每字节亚纳秒访问），现代 DRAM 设计使用并行性来增加其数据访问速率，通常称为内存访问吞吐量。

为什么 DRAM 如此慢？

下图显示了一个 DRAM 单元以及访问其内容的路径。解码器是一个电子电路，使用晶体管驱动与数千个单元的出口门连接的线。线充电或放电到达所需的水平可能需要很长时间。



更为严峻的挑战是单元驱动垂直线到达感应放大器，并允许感应放大器检测其内容。这是基于电荷共享的。门放出存储在单元中的微小电荷。如果单元内容为“1”，则微小电荷必须将长位线的大电容电位提升到足够高的水平，以触发感应放大器的检测机制。一个好的类比是让某人在长走廊的一端拿着一小杯咖啡，另一个人在走廊的另一端利用沿走廊传播的香气来确定咖啡的味道。可以通过在每个单元中使用更大、更强的电容器来加速该过程。然而，DRAM 的发展方向恰恰相反。随着时间的推移，每个单元中的电容器尺寸不断减小，因此它们的强度也在减小，以便在每个芯片中存储更多的位。这就是为什么 DRAM 的访问延迟随时间没有减少的原因。

每次访问 DRAM 位置时，都会访问包括所请求位置在内的一系列连续位置。每个 DRAM 芯片都提供了许多传感器，并且它们都并行工作。每个传感器都感知这些连续位置内的一个位的内容。一旦被传感器检测到，所有这些连续位置的数据就可以以高速传输到处理器。这些被访问和传输的连续位置被称为 DRAM 突发(bursts.)。如果应用程序专注于利用这些突发数据，那么 DRAM 将以比随机访问位置更高的速率提供数据。

意识到现代 DRAM 的突发组织，当前的 CUDA 设备采用了一种技术，允许程序员通过组织线程的内存访问方式来实现高效的全局内存访问效率。这种技术利用了一个事实，即线程束中的线程在任何给定时刻执行相同的指令。当线程束中的所有线程执行加载指令时，硬件会检测它们是否访问连续的全局内存位置。换句话说，当线程束中的所有线程访问连续的全局内存位置时，就实现了最有利的访问模式。在这种情况下，硬件将所有这些访问组合或合并成对连续 DRAM 位置的集中访问。例如，对于线程束的给定加载指令，如果线程 0 访问全局内存位置 X ，线程 1 访问位置 $X + 1$ ，线程 2 访问位置 $X + 2$ ，依此类推，所有这些访问将被合并成对连续位置的单个请求，当访问 DRAM 时(不同的 CUDA 设备可能也会对全局内存地址 X 施加对齐要求。例如，在某些 CUDA 设备中， X 要求对齐到 16 字 (即 64 字节) 边界。也就是说， X 的低 6 位应全为 0 位。由于存在第二级缓存，近期 CUDA 设备放宽了这种对齐要求。)。这种合并访问使得 DRAM 可以作为突发传送数据。(现代 CPU 在其缓存内存设计中也认识到了 DRAM 的突发组织。CPU 缓存行通常映射到一个或多个 DRAM 突发。那些充分利用每个缓存行中的字节的应用程序往往比随机访问内存位置的应用程序实现更高的性能。本章介绍的技术可以用于帮助 CPU 程序实现高性能。)

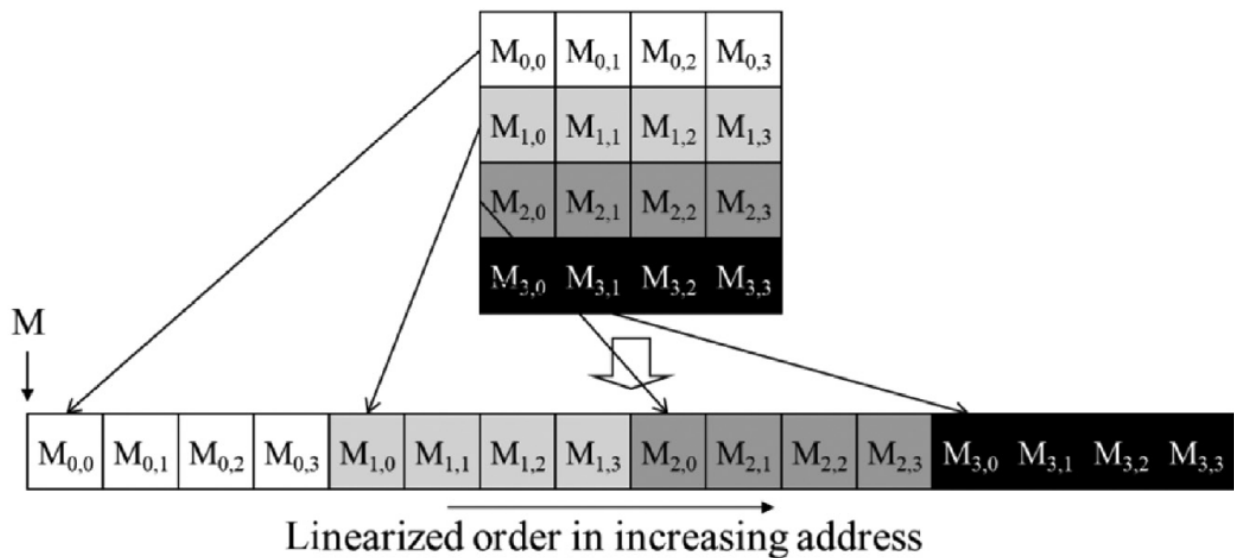


图 6.1 根据行优先顺序将矩阵元素放置到线性数组中。

要有效地使用合并硬件，我们需要回顾一下在访问 C 多维数组元素时如何形成内存地址。回想一下第三章《多维网格和数据》（为方便起见，将图 3.3 复制为图 6.1），C 和 CUDA 中的多维数组元素根据行优先约定放置在线性寻址的内存空间中。回想一下，行优先这个术语指的是数据的放置保留了行的结构：行中的所有相邻元素都放置在地址空间中的连续位置中。在图 6.1 中，第 0 行的四个元素首先按照它们在行中的出现顺序放置。然后放置第 1 行的元素，接着是第 2 行的元素，然后是第 3 行的元素。可以清楚地看出，虽然 $M_{0,0}$ 和 $M_{1,0}$ 在二维矩阵中看起来是连续的，但它们在线性寻址的内存中相隔四个位置。

假设图 6.1 中的多维数组是一个矩阵，用作矩阵乘法中的第二个输入矩阵。在这种情况下，分配给连续输出元素的线程束中的连续线程将迭代该输入矩阵的连续列。图 6.2 的左上部分显示了进行此计算的代码，右上部分显示了访问模式的逻辑视图：连续线程迭代连续列。通过检查代码，我们可以看出对 M 的访问可以被合并。数组 M 的索引是 $k * \text{Width} + \text{col}$ 。变量 k 和 Width 在线程束中的所有线程中具有相同的值。变量 col 被定义为 $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ ，这意味着连续线程（具有连续的 threadIdx.x 值）将具有连续的 col 值，因此将访问 M 的连续元素。【译注：同一个 warp 中的线程最后一维肯定是连续的，但有可能重复，这取决于 block 的维度。如果 $\text{blockDim.y} = 1$ ，也就是 block 是线性划分的，则 threadIdx.x 严格连续，比如为 4,5,6,7；如果 $\text{blockDim.y} > 1$ ，比如 $\text{blockDim.y} = 2$ ，则 threadIdx.x 会重复，比如 4 个线程为 4,5,4,5。后者其实更好，它需要读取的是相同的列。但不管怎么样，不会出现 4,5,7,8 这样的情况。】

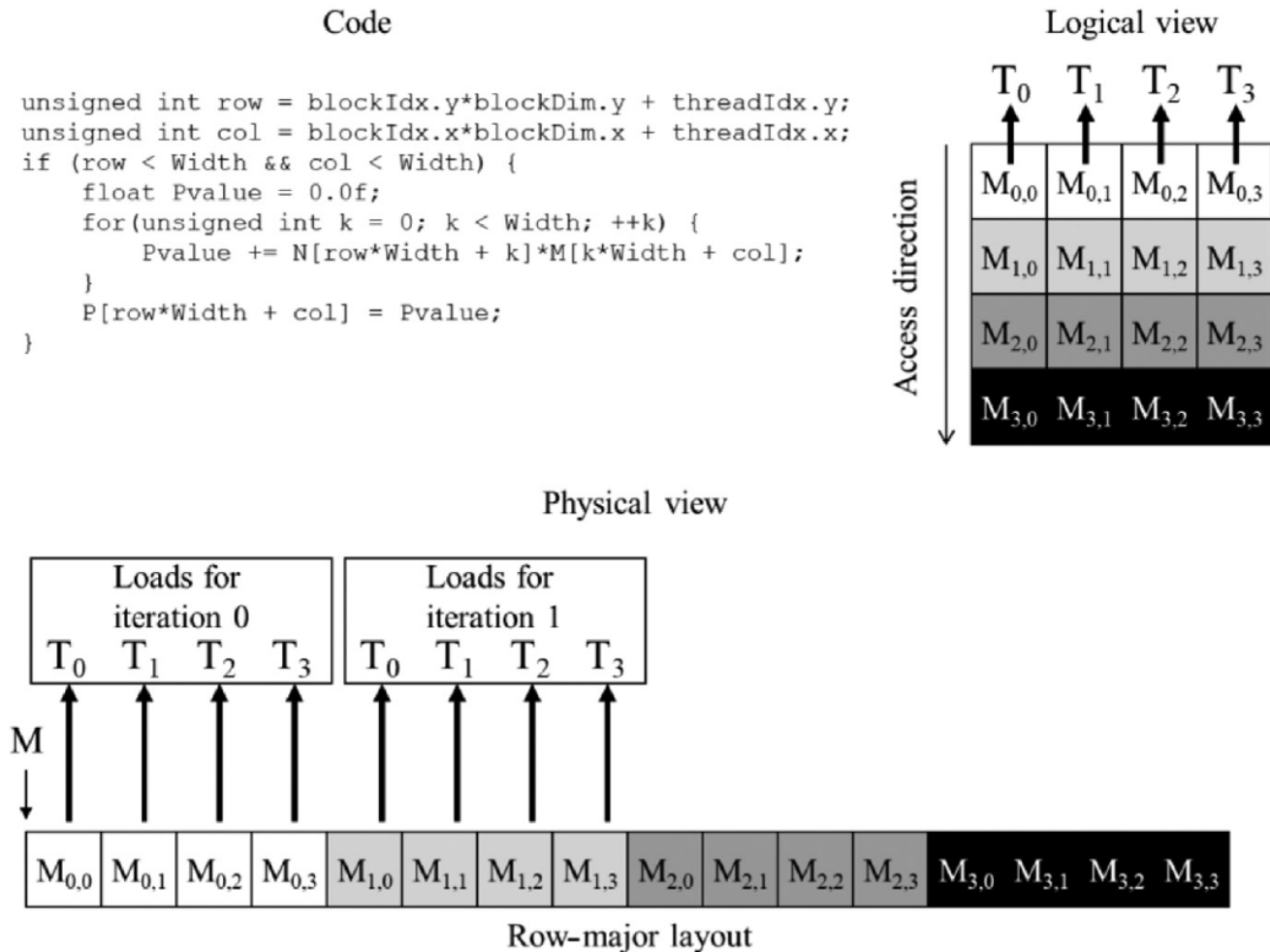


图 6.2 合并访问模式。

图 6.2 的底部显示了访问模式的物理视图。在迭代 0 中，连续线程将访问存储在内存中相邻的第 0 行的连续元素，如图 6.2 中的“迭代 0 的加载”所示。在迭代 1 中，连续线程将访问存储在内存中相邻的第 1 行的连续元素，如图 6.2 中的“迭代 1 的加载”所示。这个过程对所有行都适用。正如我们所见，线程在此过程中形成的内存访问模式是一个有利的模式，可以被合并。实际上，在我们迄今为止实现的所有内核中，我们的内存访问都是自然合并的。

现在假设矩阵以列优先顺序而不是行优先顺序存储。可能有各种各样的原因导致这种情况发生。例如，我们可能要将矩阵的转置与存储在行优先顺序的矩阵相乘。在线性代数中，我们经常需要使用矩阵的原始形式和转置形式。最好避免创建和存储两种形式。一个常见的做法是创建矩阵的一种形式，比如原始形式。当需要转置形式时，可以通过访问原始形式并交换行和列索引的角色来访问其元素。在 C 语言中，这相当于将转置矩阵视为原始矩阵的列优先布局。无论原因如何，让我们观察一下当矩阵乘法示例的第二个输入矩阵以列优先顺序存储时实现的内存访问模式。

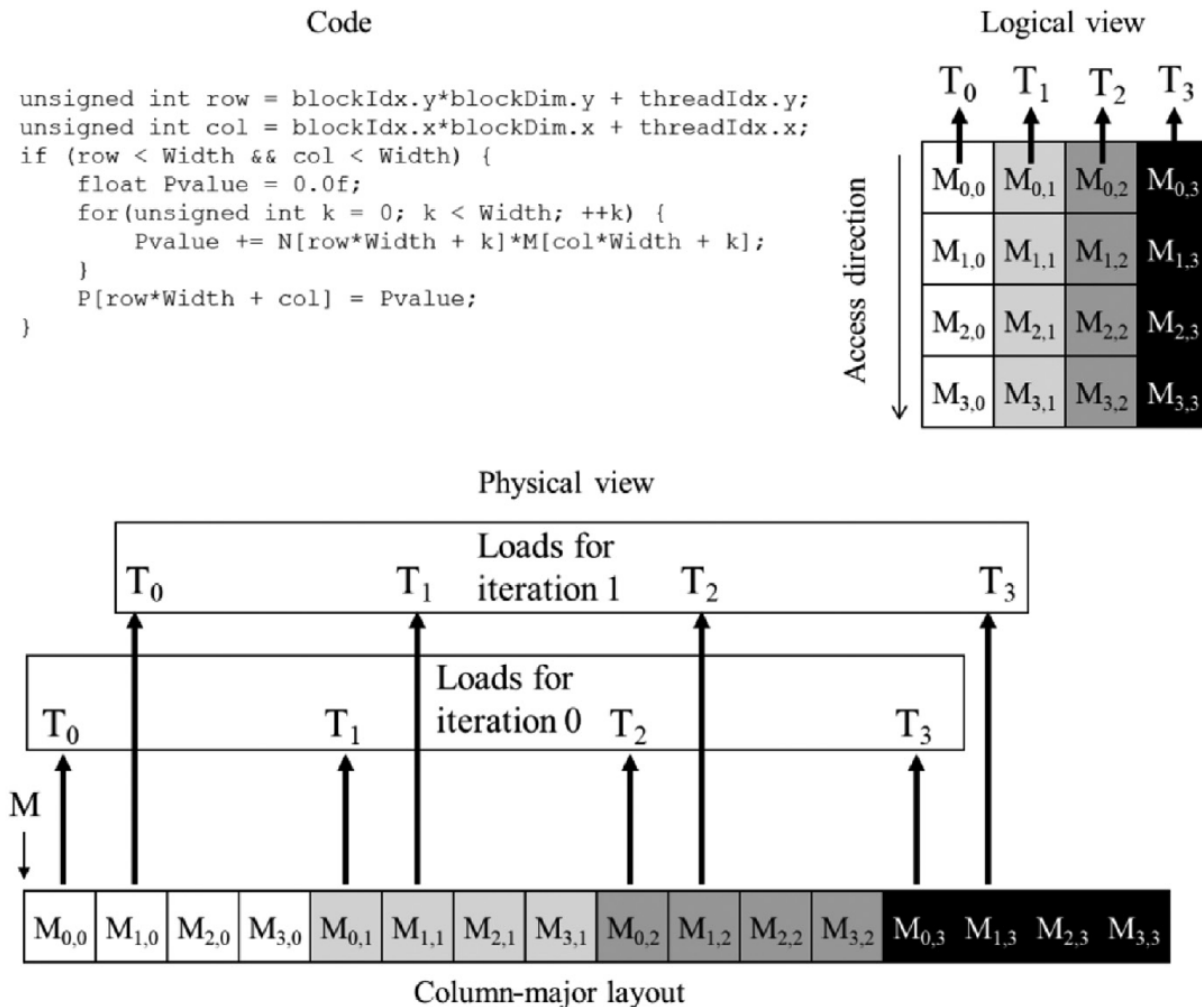


图 6.3 非合并访问模式。

图 6.3 说明了当矩阵以列优先顺序存储时，连续线程如何迭代连续的列。图 6.3 的左上部分显示了代码，右上部分显示了内存访问的逻辑视图。程序仍然试图让每个线程访问矩阵 M 的一列。通过检查代码，我们可以看出对 M 的访问不利于合并。数组 M 的索引是 $col * Width + k$ 。与之前一样， col 被定义为 $blockIdx.x * blockDim.x + threadIdx.x$ ，这意味着连续线程（具有连续的 $threadIdx.x$ 值）将具有连续的 col 值。然而，在访问 M 的索引中， col 被乘以 $Width$ ，这意味着连续线程将访问相距为 $Width$ 的 M 元素。因此，这些访问不利于合并。

在图 6.3 的底部部分，我们可以看到内存访问的物理视图与图 6.2 中的视图截然不同。在迭代 0 中，连续线程将逻辑上访问第 0 行中的连续元素，但由于列优先布局，这些元素在内存中并不相邻。这些加载被显示为图 6.3 中的“迭代 0 的加载”。类似地，在迭代 1 中，连续线程将访问在内存中也不相邻的第 1 行中的连续元素。对于一个真实的矩阵，在每个维度中通常有数百甚至数千个元素。相邻线程在每次迭代中访问的 M 元素可能相隔数百甚至数千个元素。硬件将确定这些元素的访问相距甚远，不能合并。

当计算不自然地支持内存合并时，有各种优化代码的策略。一种策略是重新安排线程与数据的映射方式；另一种策略是重新安排数据本身的布局。我们将在第 6.4 节讨论这些策略，并在本书中通过示例看到它们的应用。另一种策略是以合并的方式在全局内存和共享内存之间传输数据，并在共享内存中执行不利于合并的访问模式，以提供更快的访问延迟。我们还将看到使用这种策略的示例优化，包括我们现在将应用于以列优先布局存储的第二个输入矩阵的矩阵乘法的优化，该优化称为转角旋转(corner turning)。

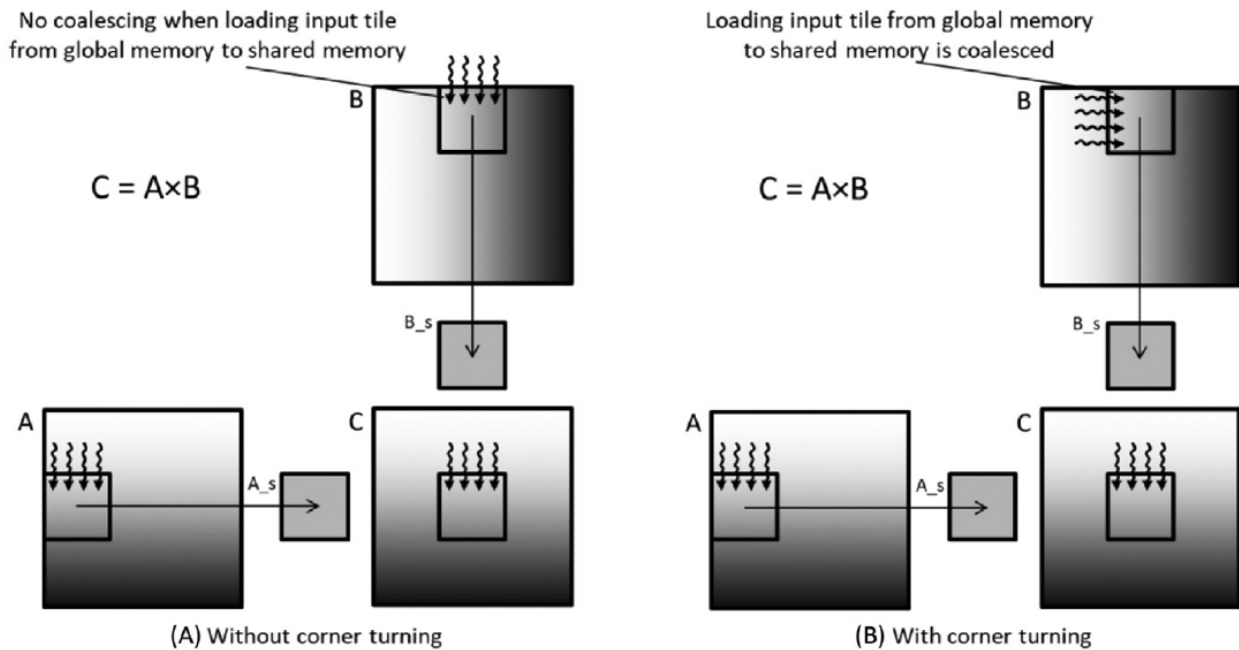


图 6.4 将角点转换应用于合并对存储在列主布局中的矩阵 B 的访问。

图 6.4 说明了转角旋转如何应用的一个示例。在这个示例中，A 是以行优先布局存储在全局内存中的输入矩阵，B 是以列优先布局存储在全局内存中的输入矩阵。它们相乘以产生以行优先布局存储在全局内存中的输出矩阵 C。该示例说明了负责输出矩阵顶部边缘的四个连续元素的四个线程如何加载输入矩阵元素。

对矩阵 A 中的输入矩阵的访问与第 5 章《内存架构和数据局部性》中的情况类似。四个线程加载输入矩阵顶部边缘的四个元素。每个线程加载一个输入元素，其在输入瓦片内的本地行和列索引与线程在输出瓦片内的输出元素相同。这些访问是合并的，因为连续线程在 A 的同一行中访问相邻的内存中的连续元素，这是按照行优先布局的要求的。

另一方面，对矩阵 B 中的输入瓦片的访问需要与第 5 章《内存架构和数据局部性》中的情况不同。图 6.4(A) 展示了如果我们采用与第 5 章《内存架构和数据局部性》相同的排列方式，访问模式将会是什么样子。即使四个线程逻辑上正在加载输入瓦片顶部边缘的四个连续元素，但是由于 B 元素的列优先布局，由连续线程加载的元素在内存中相距甚远。换句话说，负责输出瓦片中同一行中连续元素的连续线程在内存中加载非连续的位置，这导致了不合并的内存访问。

这个问题可以通过将四个连续的线程分配给加载输入瓦片中左边缘（相同列）的四个连续元素来解决，如图 6.4(B) 所示。直观地说，我们在每个线程计算加载 B 输入瓦片的线性化索引时，交换了 `threadIdx.x` 和 `threadIdx.y` 的角色。由于 B 采用列优先布局，同一列中的连续元素在内存中是相邻的。因此，连续的线程加载相邻的内存中的输入元素，这确保了内存访问是合并的。代码可以被编写为将 B 元素的瓦片以列优先布局或行优先布局放置到共享内存中。无论如何，加载了输入瓦片后，每个线程都可以几乎不受性能损失地访问其输入。这是因为共享内存采用 SRAM 技术实现，不需要合并。

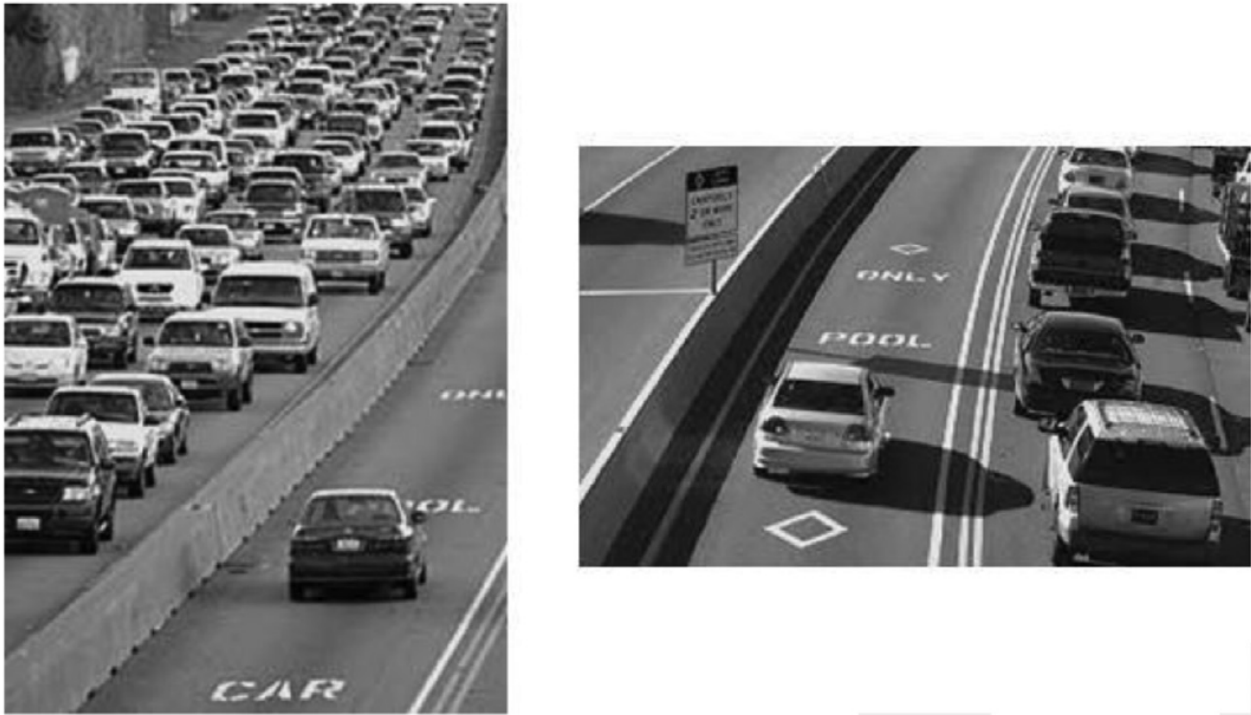


图6.5 减少高速公路系统中的交通拥堵。

内存合并的主要优势在于通过将多个内存访问合并为单个访问来减少全局内存流量。当访问同时发生且访问相邻内存位置时，访问可以合并。拥堵并不仅发生在计算中。我们大多数人都经历过公路系统的拥堵，如图 6.5 所示。公路拥堵的根本原因是太多的汽车都试图在设计用于更少车辆的道路上行驶。当拥堵发生时，每辆车的行驶时间大大增加。当交通拥堵时，通勤时间可能会翻倍甚至翻三倍。

减少交通拥堵的大多数解决方案都涉及减少道路上的汽车数量。假设通勤者的数量是恒定的，人们需要合作共乘以减少道路上的汽车数量。共乘的一种常见方式是拼车，即通勤者团体中的成员轮流驾驶一辆车去上班。政府通常需要制定政策来鼓励拼车。在某些国家，政府简单地禁止某些类别的车辆每天上路。例如，具有奇数牌照号码的车辆可能不被允许在周一、周三或周五上路。这鼓励了那些车辆在不同日子允许上路的人组成拼车团体。在其他国家，政府可能会为减少路上车辆数量的行为提供激励措施。例如，在某些国家，拥挤的高速公路的某些车道被指定为拼车车道；只有超过两三个人的车才被允许使用这些车道。还有一些国家，政府会提高汽油价格，以便人们组成拼车团以节省开支。鼓励共乘的所有措施都旨在克服共乘需要额外努力的事实，正如我们在图 6.6 中所示。

Good – people have similar schedules



Bad – people have very different schedules

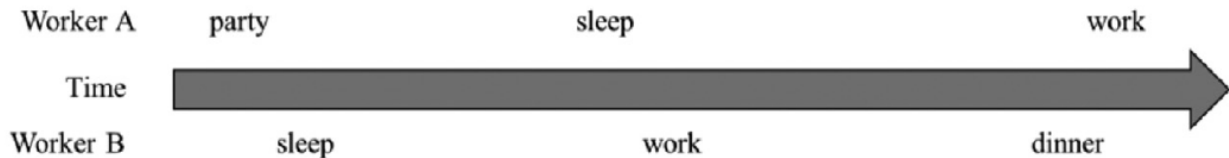


图6.6 拼车需要人们之间的同步。

共乘需要希望共乘的工作者妥协并达成共同的通勤时间表。图 6.6 的上半部分显示了共乘的良好时间表模式。时间从左到右。工作者 A 和工作者 B 在睡眠、工作和晚餐方面有类似的时间表。这使得这两个工作者可以轻松一起乘车上下班。他们相似的时间表使得他们更容易就共同出发时间和返回时间达成一致。而对于图 6.6 的下半部分显示的时间表情况则不同。在这种情况下，工作者 A 和工作者 B 的时间表非常不同。工作者 A 熬夜派对，白天睡觉，晚上去工作。工作者 B 在晚上睡觉，早上去工作，下午 6 点回家吃晚餐。这些时间表非常不同，以至于这两个工作者不可能协调出发和返回家的共同时间。内存合并与拼车安排非常相似。我们可以把数据看作通勤者，把 DRAM 访问请求看作车辆。当 DRAM 请求的速率超过 DRAM 系统提供的访问带宽时，交通拥堵就会上升，算术单元就会空闲。如果多个线程从同一 DRAM 位置访问数据，则它们可以潜在地形成一个“拼车”，将它们的访问合并为一个 DRAM 请求。然而，这需要线程具有相似的执行时间表，以便它们的数据访问可以合并为一个。处于同一个线程束中的线程是完美的候选者，因为它们通过 SIMD 执行同时执行加载指令。

6.2 隐藏内存延迟

正如我们在第 6.1 节中解释的那样，DRAM 汇聚是一种并行组织形式：在 DRAM 核心阵列中并行访问多个位置。然而，仅靠汇聚是不足以实现现代处理器所需的 DRAM 访问带宽水平的。DRAM 系统通常采用两种以上的并行组织形式：存储体(bank)和通道(channel)。在最高级别上，处理器包含一个或多个通道。每个通道都是一个内存控制器，具有将一组 DRAM 存储体连接到处理器的总线。图 6.7 展示了一个包含四个通道的处理器，每个通道都有一条总线将四个 DRAM 存储体连接到处理器。在实际系统中，处理器通常具有一个到八个通道，并且每个通道连接了大量的存储体。

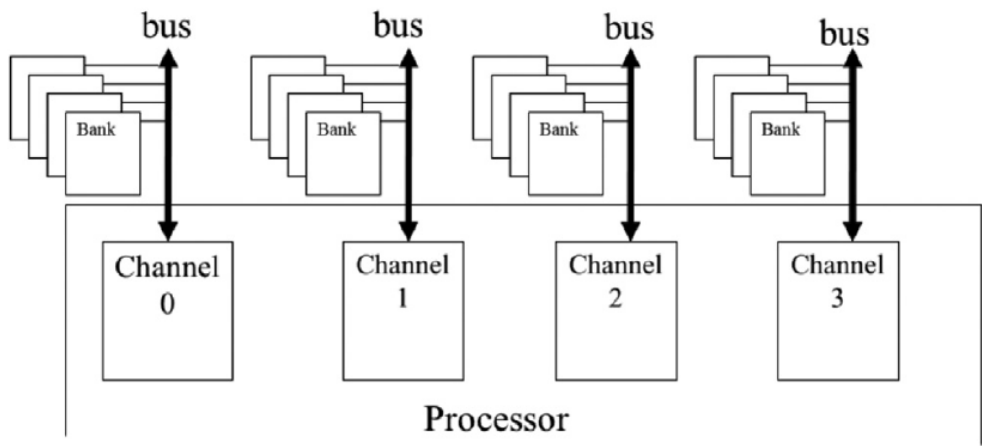


图 6.7 DRAM 系统中的通道和存储体

总线的数据传输带宽由其宽度和时钟频率定义。现代双倍数据率 (DDR) 总线每个时钟周期执行两次数据传输：在每个时钟周期的上升沿和下降沿各一次。例如，带有 1 GHz 时钟频率的 64 位 DDR 总线的带宽为 $8\text{B} \times 2 \times 1\text{GHz} = 16\text{GB/s}$ 。这似乎是一个很大的数字，但对于现代 CPU 和 GPU 来说通常太小了。现代 CPU 可能需要至少 32GB/s 的内存带宽，而现代 GPU 可能需要 256GB/s。对于此示例，CPU 将需要 2 个通道，而 GPU 将需要 16 个通道。

对于每个通道，连接到它的存储体的数量由所需的存储器数据传输带宽的存储体数量决定。这在图 6.8 中进行了说明。每个存储体包含一个 DRAM 单元阵列、用于访问这些单元的感应放大器，以及用于将数据突发传输到总线的接口（第 6.1 节）。

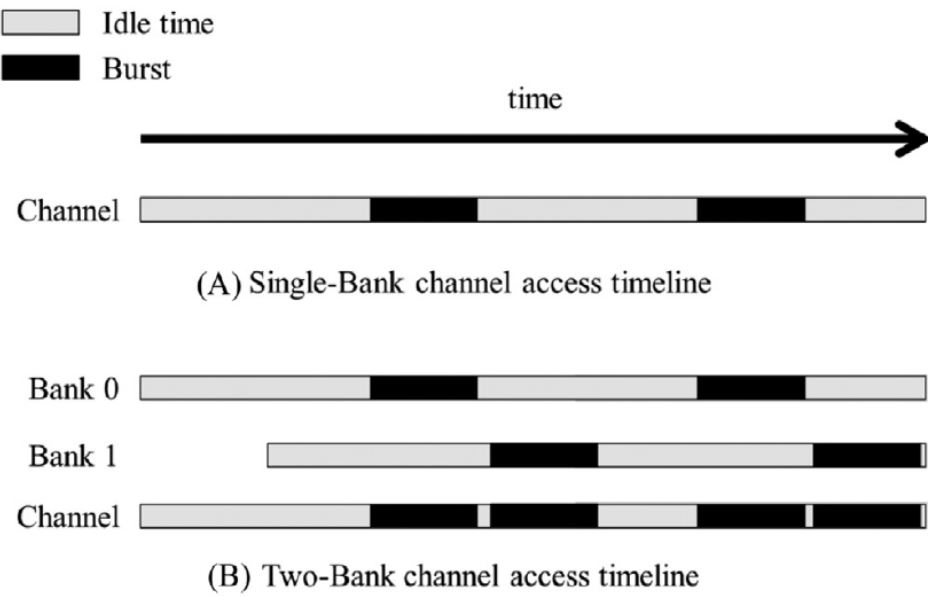


图 6.8 Banking 提高了通道数据传输带宽的利用率。

图 6.8 (A) 说明了当一个单独的存储体连接到一个通道总线时的数据传输时间。它显示了对 DRAM 存储体的两次连续内存读取访问的时间。回想一下第 6.1 节，每次访问都涉及长延迟，因为解码器要启用单元，并且单元要与感应放大器共享其存储电荷。这种延迟显示为时间范围左端的灰色部分。一旦感应放大器完成其工作，就通过总线传送突发数据。通过总线传输突发数据的时间显示为图 6.8 时间范围中左侧的黑色部分。第二个内存读取访问将在其突发数据可以传输之前遭受类似的长访问延迟（时间范围的黑色部分之间的灰色部分）。

实际上，访问延迟（灰色部分）比数据传输时间（黑色部分）长得多。应该明显地看到，单存储体组织的访问传输时间严重低估了通道总线的数据传输带宽的利用率。例如，如果 DRAM 单元阵列访问延迟与数据传输时间

的比率是 20:1，则通道总线的最大利用率将为 $1/21=4.8\%$ ；即 16GB/s 的通道将以不超过 0.76GB/s 的速率向处理器传输数据。这是完全不可接受的。这个问题是通过将多个存储体连接到一个通道总线来解决的。

当两个存储体连接到一个通道总线时，可以在第一个存储体服务另一个访问时启动对第二个存储体的访问。因此，可以重叠访问 DRAM 单元阵列的延迟。图 6.8 (B) 显示了双存储体组织的时间。我们假设存储体 0 在比图 6.8 中所示时间窗口更早的时间开始了。在第一个存储体开始访问其单元阵列后不久，第二个存储体也开始访问其单元阵列。当存储体 0 的访问完成后，它将传输突发数据（时间范围中最左侧的黑色部分）。一旦存储体 0 完成其数据传输，存储体 1 就可以传输其突发数据（第二黑色部分）。这种模式对于下一次访问重复。

从图 6.8 (B) 中，我们可以看到，通过具有两个存储体，我们可以潜在地将通道总线的数据传输带宽利用率加倍。一般来说，如果 DRAM 单元阵列访问延迟和数据传输时间的比率为 R ，那么我们需要至少 $R+1$ 个存储体，如果我们希望充分利用通道总线的数据传输带宽。例如，如果比率为 20，那么我们将需要至少将 21 个存储体连接到每个通道总线。一般来说，连接到每个通道总线的存储体的数量需要大于 R ，原因有两个。一个是具有更多的存储体可以减少多个同时访问目标相同存储体的概率，这种现象称为存储体冲突。由于每个存储体一次只能服务一个访问，因此对于这些冲突访问，单元阵列访问延迟无法重叠。拥有更多的存储体增加了这些访问将分布在多个存储体之间的概率。第二个原因是每个单元阵列的大小被设置为实现合理的延迟和可制造性。这限制了每个存储体可以提供的单元数。为了能够支持所需的内存大小，可能需要许多存储体。

线程的并行执行与 DRAM 系统的并行组织之间存在重要的联系。为了实现设备指定的内存访问带宽，必须有足够数量的线程进行同时内存访问。这一观察反映了最大化占用率的另一个好处。回想一下第 4 章《计算体系结构与调度》中，我们看到最大化占用率可以确保在流多处理器 (SM) 上有足够数量的线程驻留，以隐藏核心流水线延迟，从而有效地利用指令吞吐量。正如我们现在看到的，最大化占用率还有一个额外的好处，即确保发出足够的内存访问请求以隐藏 DRAM 访问延迟，从而有效地利用内存带宽。当然，为了实现最佳的带宽利用率，这些内存访问必须均匀分布在通道和存储体之间，每次访问存储体也必须是一个集合的访问。

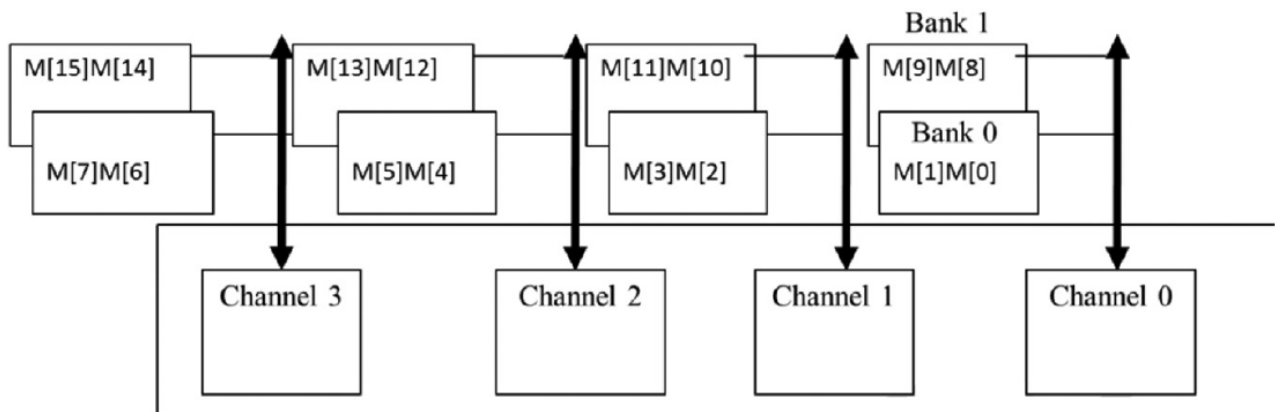


图 6.9 将数组元素分配到通道和存储区。

图 6.9 展示了将数组 M 的元素分配到通道和存储体的一个玩具示例。我们假设一个小的突发大小为两个元素（8 字节）。分配是通过硬件设计完成的。通道和存储体的寻址使得数组的前 8 字节 ($M[0]$ 和 $M[1]$) 存储在通道 0 的存储体 0 中，接下来的 8 字节 ($M[2]$ 和 $M[3]$) 存储在通道 1 的存储体 0 中，接下来的 8 字节 ($M[4]$ 和 $M[5]$) 存储在通道 2 的存储体 0 中，以及接下来的 8 字节 ($M[6]$ 和 $M[7]$) 存储在通道 3 的存储体 0 中。

此时，分配将回到通道 0，但将使用存储体 1 用于接下来的 8 字节 ($M[8]$ 和 $M[9]$)。因此，元素 $M[10]$ 和 $M[11]$ 将在通道 1 的存储体 1 中， $M[12]$ 和 $M[13]$ 将在通道 2 的存储体 1 中， $M[14]$ 和 $M[15]$ 将在通道 3 的存储体 1 中。虽然图中未显示，但任何额外的元素将被环绕并从通道 0 的存储体 0 开始。例如，如果还有更多的元素， $M[16]$ 和 $M[17]$ 将存储在通道 0 的存储体 0 中， $M[18]$ 和 $M[19]$ 将存储在通道 1 的存储体 0 中，依此类推。

在图6.9中所示的分布方案，通常称为交错数据分布，将元素分布到系统的各个存储区和通道中。这种方案确保即使是相对较小的数组也能很好地分布。因此，我们只分配足够的元素以充分利用通道0的存储区0的DRAM突发，然后再转移到通道1的存储区0。在我们的示例中，只要有至少16个元素，分布就会涉及存储所有通道和存储区的元素。

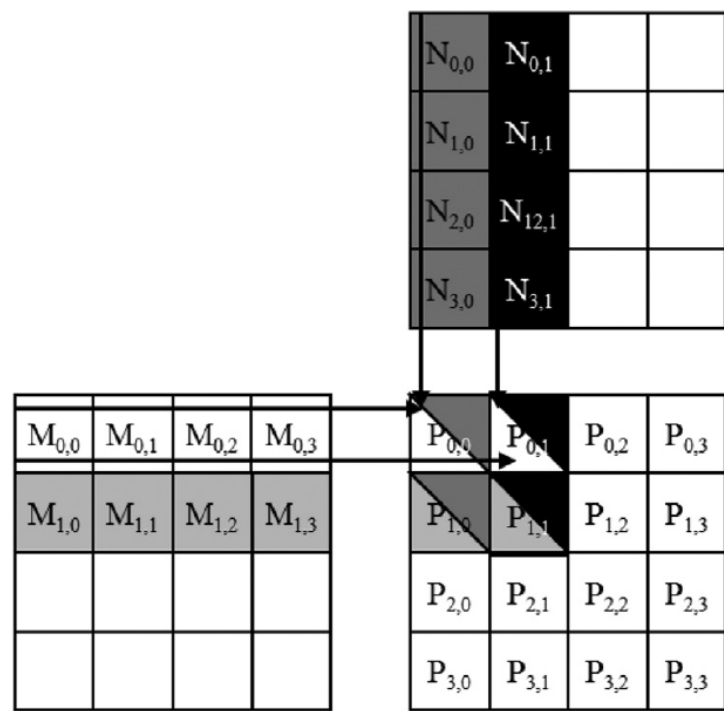


图 6.10 一个矩阵乘法的小例子（复制自图5.5）。

Tiles loaded by	Block 0,0	Block 0,1	Block 1,0	Block 1,1
Phase 0 (2D index)	M[0][0],M[0][1], M[1][0], M[1][1]	M[0][0], M[0][1], M[1][0], M[1][1]	M[2][0], M[2][1], M[3][0], M[3][1] 应该是M[9]	M[2][0], M[2][1], M[3][0], M[3][1]
Phase 0 (linearized index)	M[0], M[1], M[4], M[5]	M[0], M[1], M[4], M[5]	M[8], M[9] , M[12], M[13]	M[8], M[9], M[12], M[13]
Phase 1 (2D index)	M[0][2],M[0][3], M[1][2], M[1][3]	M[0][2],M[0][3], M[1][2], M[1][3]	M[2][2], M[2][3], M[3][2], M[3][3]	M[2][2], M[2][3], M[3][2], M[3][3]
Phase 1 (linearized index)	M[2], M[3], M[6], M[7]	M[2], M[3], M[6], M[7]	M[10], M[11], M[14], M[15]	M[10], M[11], M[14], M[15]

图 6.11 每个阶段由线程块加载的 M 元素。

我们现在说明并行线程执行与并行内存组织之间的交互。我们将使用图5.5中的示例，复制为图6.10。我们假设乘法将使用2x3x2线程块和2x3x2瓦片进行。

在内核执行的阶段0期间，所有四个线程块将加载它们的第一个瓦片。图6.11中显示了每个瓦片中涉及的M元素。第2行显示了阶段0中访问的M元素及其2D索引。第3行显示了相同的M元素及其线性化索引。假设所有线

程块都是并行执行的。我们可以看到每个块将进行两次组合访问。

根据图6.9中的分布，这些组合访问将分别发送到通道0的两个存储区以及通道2的两个存储区。这四个访问将并行执行，以利用两个通道并提高每个通道数据传输带宽的利用率。

我们还可以看到Block0,0和Block 0,1将加载相同的M元素。大多数现代设备都配备了缓存，只要这些块的执行时间足够接近，缓存将将这些访问合并为一个。事实上，GPU设备中的缓存存储器主要设计用于合并这些访问并减少对DRAM系统的访问次数。

第4行和第5行显示了内核执行阶段1期间加载的M元素。我们可以看到现在的访问是针对通道1和通道3中的存储区的。这些访问将再次并行执行。读者应该清楚地看到，并行线程执行和DRAM系统的并行结构之间存在着共生关系。一方面，良好利用DRAM系统潜在的访问带宽需要许多线程同时访问DRAM中的数据。另一方面，设备的执行吞吐量依赖于对DRAM系统的并行结构，即存储区和通道的良好利用。例如，如果同时执行的线程都访问同一通道中的数据，那么内存访问吞吐量和整体设备执行速度将大大降低。

读者被邀请验证，例如通过相同的2x3x2线程块配置来乘以两个较大的矩阵，例如8x3x8，将利用图6.9中的所有四个通道。另一方面，增加DRAM突发大小将需要乘以更大的矩阵才能充分利用所有通道的数据传输带宽。

6.3 线程粗化(thread coarsening)

到目前为止，我们所见到的所有内核都是在最细粒度上将工作并行化的。也就是说，每个线程被分配了最小可能的工作单元。例如，在向量加法内核中，每个线程被分配了一个输出元素。在RGB到灰度转换和图像模糊内核中，每个线程被分配了输出图像中的一个像素。在矩阵乘法内核中，每个线程被分配了输出矩阵中的一个元素。

在最细粒度上将工作并行化的优点在于，它增强了透明可伸缩性，正如第4章《计算架构和调度》中所讨论的那样。如果硬件有足够的资源来并行执行所有工作，那么应用程序已经暴露了足够的并行性来充分利用硬件。否则，如果硬件没有足够的资源来并行执行所有工作，硬件可以简单地通过依次执行线程块来串行化工作。

在最细粒度上将工作并行化的缺点在于，当并行化工作的代价很高时。并行性的代价可以采用许多形式，例如不同线程块重复加载数据、冗余工作、同步开销等。当硬件并行执行线程时，通常值得支付这种并行性的代价。然而，如果由于资源不足而导致硬件串行化工作，那么这种代价就是不必要的。在这种情况下，程序员部分串行化工作并减少支付的并行性代价会更好。这可以通过将每个线程分配多个工作单元来实现，通常被称为线程粗化。

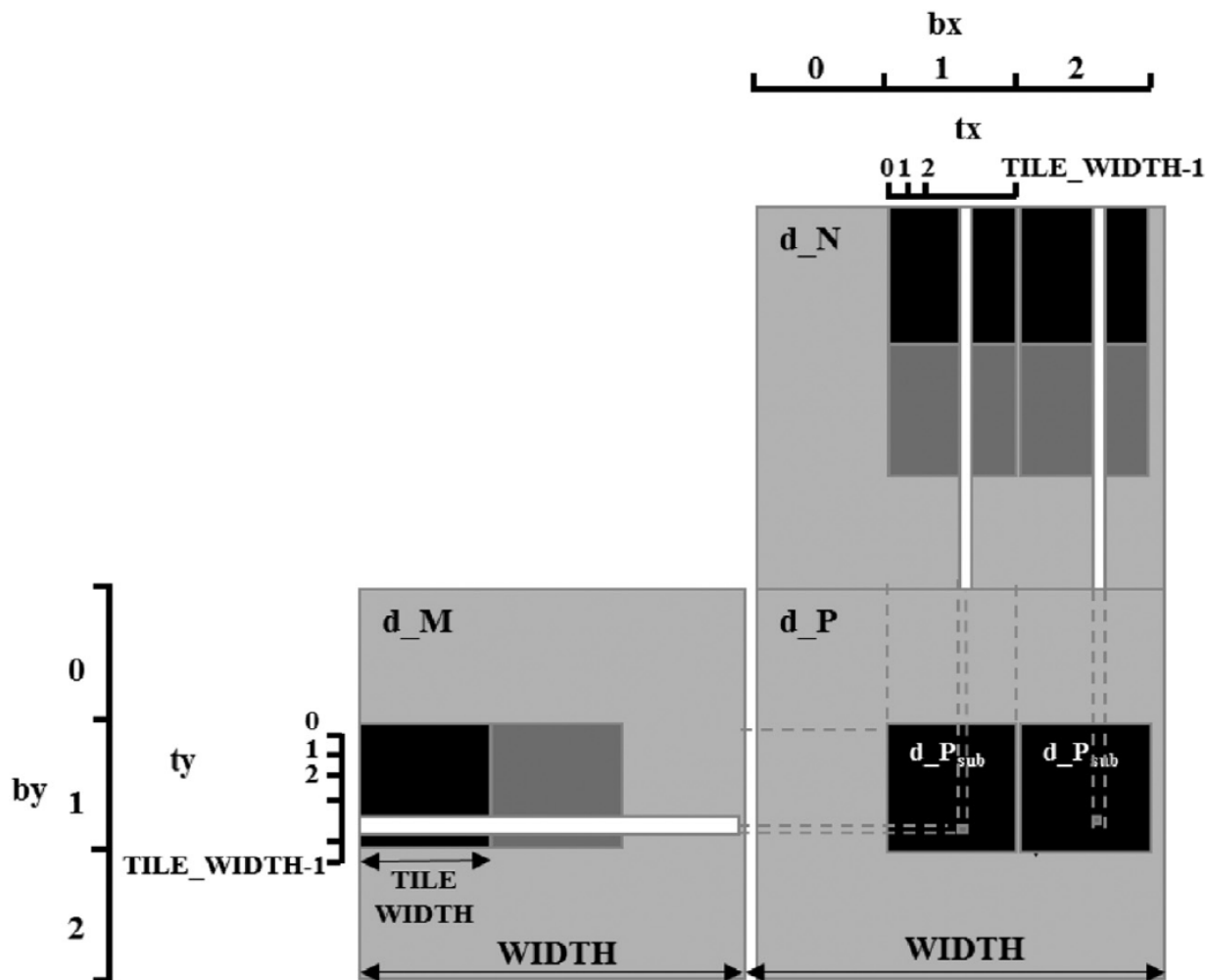


图 6.12 分块矩阵乘法的线程粗化。

我们使用第 5 章《内存架构和数据局部性》中的分块矩阵乘法示例来演示线程粗化优化。图 6.12 描述了计算输出矩阵 P 的两个水平相邻输出块的存储器访问模式。对于这些输出块中的每一个，我们观察到矩阵 N 的不同输入块需要被加载。然而，矩阵 M 的相同输入块被同时加载到了这两个输出块中。

在第 5 章《内存架构和数据局部性》中的分块实现中，每个输出块由不同的线程块处理。由于共享内存内容不能在块之间共享，因此每个块必须加载矩阵 M 的自己的副本。尽管让不同的线程块加载相同的输入块是多余的，但这是我们为了能够使用不同的块并行处理两个输出块而支付的代价。如果这些线程块并行运行，这个代价可能是值得支付的。另一方面，如果这些线程块由硬件串行执行，这个代价就是徒劳的。在后一种情况下，对程序员来说，最好是让一个线程块处理两个输出块，其中每个线程在块中处理两个输出元素。这样，粗化的线程块将加载矩阵 M 的输入块一次，并重复使用它们用于多个输出块。

```

01  #define TILE_WIDTH    32
02  #define COARSE_FACTOR 4
03  __global__ void matrixMulKernel(float* M, float* N, float* P, int width)
04  {
05      __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
06      __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
07
08      int bx = blockIdx.x;  int by = blockIdx.y;
09      int tx = threadIdx.x; int ty = threadIdx.y;
10
11      // Identify the row and column of the P element to work on
12      int row = by*TILE_WIDTH + ty;
13      int colStart = bx*TILE_WIDTH*COARSE_FACTOR + tx;
14
15      // Initialize Pvalue for all output elements
16      float Pvalue[COARSE_FACTOR];
17      for(int c = 0; c < COARSE_FACTOR; ++c) {
18          Pvalue[c] = 0.0f;
19      }
20
21      // Loop over the M and N tiles required to compute P element
22      for(int ph = 0; ph < width/TILE_WIDTH; ++ph) {
23
24          // Collaborative loading of M tile into shared memory
25          Mds[ty][tx] = M[row*width + ph*TILE_WIDTH + tx];
26
27          for(int c = 0; c < COARSE_FACTOR; ++c) {
28
29              int col = colStart + c*TILE_WIDTH;
30
31              // Collaborative loading of N tile into shared memory
32              Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*width + col];
33              __syncthreads();
34
35              for(int k = 0; k < TILE_WIDTH; ++k) {
36                  Pvalue[c] += Mds[ty][k]*Nds[k][tx];
37              }
38              __syncthreads();
39          }
40      }
41
42  }
43
44  for(int c = 0; c < COARSE_FACTOR; ++c) {
45      int col = colStart + c*TILE_WIDTH;
46      P[row*width + col] = Pvalue[c];
47  }
48
49  }

```

图 6.13 分块矩阵乘法的线程粗化代码。

图 6.13 展示了如何将线程粗化应用于第 5 章《内存架构和数据局部性》中的分块矩阵乘法代码。第 2 行添加了一个常量 `COARSE_FACTOR`，表示粗化因子，即每个粗化线程负责的原始工作单位数。第 13 行将列索引的初始化替换为 `colStart` 的初始化，因为现在线程负责的不同列索引的元素有多个。在计算 `colStart` 时，块索引 `bx` 被乘以 `TILE_WIDTH x COARSE_FACTOR`，而不仅仅是 `TILE_WIDTH`，因为每个线程块现在负责 `TILE_WIDTH x COARSE_FACTOR` 列。在第 16 到 19 行，声明并初始化了多个 `Pvalue` 实例，每个实例对应于粗化线程负责的每个元素。第 17 行的循环遍历粗化线程负责的不同工作单位，有时被称为粗化循环。在第 22 行的循环内，遍历输入块，每次循环只加载一个 `M` 的块，与原始代码相同。然而，对于每个加载的 `M` 块，粗化循环在第 27 行加载和使用多个 `N` 块。此循环首先确定粗化线程负责的当前块的列（第 29 行），然后加载 `N` 块（第 32 行）并

用于计算和更新每次迭代的不同 Pvalue (第 35 到 37 行) 。最后 , 在第 44 到 47 行 , 另一个粗化循环用于让每个粗化线程更新其负责的输出元素。

【译注：由于一个线程复杂的列是原来的COARSE_FACTOR倍，因此在调用时，需要调整gridDim。比如原来是这样的：

```
// Launch kernel
dim3 blockDim(TILE_WIDTH, TILE_WIDTH);
dim3 gridDim(ceil((float)numCColumns / blockDim.x), ceil((float)numCRows /
blockDim.y));
```

需要调整为：

```
// Launch kernel
dim3 blockDim(TILE_WIDTH, TILE_WIDTH);
dim3 gridDim(ceil((float)numCColumns / (blockDim.x * COARSE_FACTOR)),
ceil((float)numCRows / blockDim.y));
```

】

线程粗化是一种强大的优化方法，可以为许多应用程序带来显著的性能改进。这是一种常见的优化方法。然而，在应用线程粗化时有几个需要避免的陷阱。首先，必须小心不要在不必要时应用该优化。回想一下，当并行化存在成本时，通过粗化来减少成本是有益的，比如多余的数据加载、多余的工作、同步开销等。并不是所有的计算都有这样的成本。例如，在第 2 章《异构数据并行计算》中的向量加法内核中，并行处理不同的向量元素不会产生成本。因此，预计将线程粗化应用于向量加法内核不会产生实质性的性能差异。相同的情况也适用于第 3 章《多维网格和数据》中的 RGB 到灰度转换内核。

第二个要避免的陷阱是不要应用太多的粗化，以至于硬件资源被低效利用。回想一下，向硬件暴露尽可能多的并行性可以实现透明的可伸缩性。它为硬件提供了并行化或串行化工作的灵活性，具体取决于其拥有的执行资源量。当程序员粗化线程时，他们减少了向硬件暴露的并行性量。如果粗化因子过高，将向硬件暴露的并行性不足，导致一些并行执行资源被未利用。在实践中，不同的设备具有不同数量的执行资源，因此最佳的粗化因子通常是设备特定和数据集特定的，并且需要针对不同的设备和数据集进行重新调整。因此，当应用线程粗化时，可伸缩性变得不太透明。

应用线程粗化的第三个陷阱是避免增加资源消耗，以至于影响占用率。根据内核的不同，线程粗化可能需要每个线程使用更多的寄存器或每个线程块使用更多的共享内存。如果是这种情况，程序员必须小心不要使用过多的寄存器或过多的共享内存，以至于降低占用率。降低占用率可能带来的性能惩罚可能比线程粗化带来的性能好处更大。

6.4 优化检查表

在本书的第一部分中，我们介绍了CUDA程序员常用的各种优化方法，以提高其代码的性能。我们将这些优化整合到一个单一的检查表中，如表6.1所示。这个检查表并不是详尽无遗的，但它包含了许多通用的优化方法，这些优化方法在不同的应用程序中都很常见，程序员应该首先考虑。在本书的第二和第三部分中，我们将把这些优化方法应用到各种并行模式和应用中，以了解它们在不同情境下的运作方式。在本节中，我们将对每种优化进行简要回顾，并讨论应用它的策略。

表 6.1 优化清单

优化方法	对于计算核心的提升	对于内存的提升	策略
最大化占用率	更多的工作来隐藏pipeline延迟	更多并行内存访问以隐藏DRAM	调整使用SM资源，如每个块的线程数量，每个块的共享内存，以及每个线程的寄存器。
启用合并的全局内存访问	等待全局内存访问的管道停顿更少	全局内存流量减少，突发/缓存行的利用率更高。	以协调的方式在全局内存和共享内存之间传输，并在共享内存中执行不协调的访问（例如，corner turning）。重新安排线程对数据的映射重新安排数据的布局
减少控制分支	高SIMD效率（SIMD执行期间较少空闲核心）	-	重新安排线程对工作和/或数据的映射 重新排列数据的布局
数据重用的分块	减少等待全局内存访问的流水线停顿	更少的全局内存访问量	将在块内重复使用的数据放置在共享内存或寄存器中，以便它只在全局内存和SM之间传输一次
私有化（稍后介绍）	减少等待原子更新的流水线停顿	更少的原子更新竞争和串行化	对数据的私有副本应用部分更新，然后在完成时更新通用副本
线程粗化	减少冗余工作、分支或同步	更少的冗余全局内存访问	将多个并行性单位分配给每个线程，以减少不必要产生的并行性成本

表6.1中的第一个优化是最大化SM上线程的利用率。这个优化是在第4章中介绍的，该章强调了拥有比核心多得多的线程的重要性，以确保在核心管道中有足够的工作可用来隐藏长延迟操作。为了最大化利用率，程序员可以调整其内核的资源使用，以确保每个SM允许的最大块数或寄存器数不会限制同时分配给SM的线程数。在第5章中，共享内存被介绍为另一种应该仔细调整使用的资源，以免限制利用率。在本章中，强调了最大化利用率的重要性，这不仅是为了隐藏核心管道延迟，还包括隐藏内存延迟。同时执行许多线程确保生成足够的内存访问以充分利用内存带宽。

表6.1中的第二个优化是通过确保同一warp中的线程访问相邻内存位置来使用合并的全局内存访问。这个优化是在本章中介绍的，其中强调了硬件将对相邻内存位置的访问合并为单个内存请求的能力，作为减少全局内存流量并提高DRAM突发利用率的方法。到目前为止，在本书的这一部分中，我们所看到的内核自然展示了合并访问。然而，在第二和第三部分的书中，我们将看到许多例子，其中内存访问模式更加不规则，因此需要更多的努力来实现合并。

在具有不规则访问模式的应用程序中，可以采用多种策略来实现合并。一种策略是以合并的方式从全局内存加载数据到共享内存中，然后在共享内存上执行不规则的访问。我们已经在本章中看到了这种策略的一个例子，即角点转换。我们将在第12章中看到另一个这种策略的例子，即合并模式。在这个模式中，同一块中的线程需要在相同的数组中执行二进制搜索，因此它们合作将该数组以合并的方式从全局内存加载到共享内存中，然后每个线程在共享内存中执行二进制搜索。我们还将在第13章中看到这种策略的另一个例子，即排序模式。在这

个模式中，线程以分散的方式写出结果，因此它们可以合作在共享内存中执行它们的分散访问，然后将结果从共享内存写入到具有更多合并访问的全局内存中的相邻目的地。

在具有不规则访问模式的应用程序中实现合并的另一种策略是重新安排线程与数据元素的映射方式。我们将在第10章中看到这种策略的一个例子，即减少和最小化分歧，它涵盖了减少模式。在具有不规则访问模式的应用程序中实现合并的另一种策略是重新排列数据本身的布局方式。我们将在第14章中看到这种策略的一个例子，即稀疏矩阵计算，它涵盖了稀疏矩阵计算和存储格式，特别是在讨论ELL和JDS格式时。

表6.1中的第三个优化是最小化控制分歧。控制分歧是在第4章中介绍的，强调了同一个warp中的线程采用相同的控制路径的重要性，这是确保在SIMD执行期间所有核心都得到有效利用的一种方法。到目前为止，在本书的这一部分中，我们所看到的内核没有表现出控制分歧，除了在边界条件下不可避免的分歧。然而，在第二和第三部分的书，我们将看到许多例子，其中控制分歧可能对性能造成重大损害。

最小化控制分歧的多种策略可供选择。一种策略是重新安排工作和/或数据在线程之间的分配，以确保一个warp中的线程在其他warp中的线程之前全部使用。我们将在第10章中看到这种策略的一个例子，即减少和最小化分歧，它涵盖了减少模式，以及第11章中的前缀和（扫描），它涵盖了扫描模式。重新安排工作和/或数据在线程之间的分配也可以用来确保同一warp中的线程具有相似的工作负载。我们将在第15章中看到这种策略的一个例子，即图遍历，它涵盖了图遍历，在该章中，我们将讨论顶点中心和边中心并行化方案之间的权衡。最小化控制分歧的另一种策略是重新安排数据的布局方式，以确保处理相邻数据的同一warp中的线程具有相似的工作负载。我们将在第14章中看到这种策略的一个例子，即稀疏矩阵计算，它涵盖了稀疏矩阵计算和存储格式，特别是在讨论JDS格式时。

表6.1中的第四个优化是通过将重复使用的数据分块存储在共享内存或寄存器中，并重复从中访问，从而最大程度地减少了在全局内存和SM之间传输的次数。分块是在第5章中介绍的，在矩阵乘法的上下文中，该章中，处理相同输出块的线程协作地将相应的输入块加载到共享内存中，然后从共享内存中重复访问这些输入块。在本书的第二和第三部分中，我们将再次看到这种优化应用在大多数并行模式中。我们将观察到当输入和输出块具有不同尺寸时，应用分块的挑战。这个挑战出现在涵盖卷积模式的第7章和涵盖模板模式的第8章中。我们还将观察到数据块可以存储在寄存器中，而不仅仅是共享内存中。这种观察在涵盖模板模式的第8章中最为明显。我们还将观察到，分块适用于重复访问的输出数据，而不仅仅是输入数据。

表6.1中的第五个优化是私有化。这个优化还没有介绍，但我们在这里提及它是为了完整性。私有化涉及到多个线程或块需要更新通用输出的情况。为了避免同时更新相同数据的开销，可以创建数据的私有副本并部分更新，然后在完成时可以从私有副本向通用副本进行最终更新。我们将在涵盖直方图模式的第9章中看到这种优化的一个例子，在该章中，多个线程需要更新相同的直方图计数器。我们还将涵盖图遍历的第15章中看到这种优化的一个例子，在该章中，多个线程需要将条目添加到同一队列中。

表6.1中的第六个优化是线程合并，其中多个并行单元被分配给一个线程，以降低并行化的代价，如果硬件本来要串行化线程的话。线程合并是在本章中引入的，在分块矩阵乘法的上下文中，多个处理相邻输出块的线程块重复加载相同的输入块，这导致了并行化的代价。在这种情况下，将一个线程块分配给处理多个相邻输出块可以使得输入块只需加载一次。在本书的第二和第三部分中，我们将看到线程合并应用在不同的上下文中，每次的并行化代价都不同。在涵盖模板模式的第8章中，线程合并被应用于减少输入数据的冗余加载，就像在本章中一样。在涵盖直方图模式的第9章中，线程合并有助于减少需要提交给通用副本的私有副本的数量。在涵盖减少模式的第10章和扫描模式的第11章中，线程合并被用来减少同步和控制分歧的开销。同样在涵盖扫描模式的第11章中，线程合并还有助于减少与顺序算法相比并行算法执行的冗余工作。在涵盖合并模式的第12章中，线程合并减少了需要执行的二分搜索操作的数量，以确定每个线程的输入段。在涵盖排序模式的第13章中，线程合并有助于改善内存合并。再次强调，表6.1中的检查表并不旨在详尽无遗，但它包含了跨不同计算模式常见的主要优化类型。这些优化出现在本书的第二和第三部分的多个章节中。我们还将看到其他出现在特定章节中的优

化。例如，在涵盖卷积模式的第7章中，我们将介绍使用常量内存。在涵盖减少模式的第10章中，我们将介绍双缓冲优化。

6.5 了解计算的瓶颈

在决定对特定计算应用哪种优化时，首先要了解限制该计算性能的资源是什么。通常，限制计算性能的资源被称为性能瓶颈。优化通常会使用更多的一种资源来减轻对另一种资源的负担。如果应用的优化不针对瓶颈资源，那么可能不会从优化中获益。更糟糕的是，优化尝试甚至可能损害性能。

例如，共享内存平铺(tiling)增加了对共享内存的使用，以减轻全局内存带宽的压力。当瓶颈资源是全局内存带宽且所加载的数据被重用，这种优化非常有效。然而，如果性能受到占用率的限制，并且占用率已经受到使用过多共享内存的限制，那么应用共享内存平铺很可能会使情况变得更糟。

要了解计算性能的限制资源是什么，GPU 计算平台通常提供各种性能分析工具。我们建议读者查阅 CUDA 文档，以获取有关如何使用性能分析工具来识别其计算性能瓶颈的更多信息 ([NVIDIA, Profiler](#))。性能瓶颈可能是硬件特定的，这意味着同一计算在不同设备上可能遇到不同的瓶颈。因此，识别性能瓶颈并应用性能优化的过程需要对 GPU 架构和不同 GPU 设备之间的架构差异有很好的理解。

6.6 总结

在本章中，我们介绍了 GPU 的片外内存 (DRAM) 架构，并讨论了相关的性能考虑因素，如全局内存访问的合并和利用内存并行性隐藏内存延迟。然后，我们介绍了一个重要的优化：线程粒度粗化。通过本章和之前章节所提供的见解，读者应该能够推断出他们遇到的任何内核代码的性能。我们通过呈现一个常用性能优化的检查表来结束本书的这一部分，这些优化被广泛用于优化许多计算。在本书的下两部分中，我们将继续研究这些优化在并行计算模式和应用案例研究中的实际应用。